Research Article Vol. 16, No. 4 / April 2024 / *Journal of Optical Communications and Networking* 449

JOURNAL OF
Optical Communications
and Networking

Large-scale quantum key distribution network simulator

Emir Dervisevic,[1,*] **Miroslav Voznak,**[2] and **Miralem Mehic**[1,2]

[1]*Department of Telecommunications, Faculty of Electrical Engineering, University of Sarajevo, Zmaja od Bosne bb, 71000 Sarajevo, Bosnia and Herzegovina*
[2]*VSB–Technical University of Ostrava, 17. listopadu 2172/15, 708 00 Ostrava, Czechia*
**emir.dervisevic@etf.unsa.ba*

Received 14 August 2023; revised 11 February 2024; accepted 21 February 2024; published 12 March 2024

The wide range of supported services in modern telecommunication networks has increased the demand for highly secure means of communication. Common security frameworks based on the computational security model are expected to become insecure due to significant advances in quantum computing. Quantum key distribution (QKD), a new secret key agreement primitive, enables long-anticipated practical information-theoretical security (ITS). Over the past two decades, academic and industrial communities have devoted their time and resources to developing QKD-based networks that can distribute and serve ITS keys to remote parties. However, because the availability of QKD network testbeds to the larger research community is limited and the deployment of such systems is costly and difficult, progress in this area is noticeably slow. To address this problem and spur future development and education, we provide a valuable, unique tool for simulating a QKD network. The tool is essential to testing novel network management methodologies applied to large-scale QKD networks. The simulator model contained in the tool was validated by simulating a network with six nodes and three pairs of users. The results indicate that the designed functional elements operate correctly. © 2024 Optica Publishing Group

https://doi.org/10.1364/JOCN.503356

1. INTRODUCTION

In contrast to popular, computationally secure cryptographic solutions for the fundamental task of modern cryptography (secret key agreement), quantum key distribution (QKD) offers information-theoretic security (ITS) [1]. It has no true competitor and has provided the groundwork for practical ITS communications combined with Vernam, the well-known one-time pad (OTP) cipher [2,3]. QKD requires a logical QKD link, which consists of a direct physical (optical) connection, i.e., a quantum channel, and an authenticated public channel. The quantum channel is used for the transmission of a random sequence of bits encoded in the nonorthogonal states of a quantum system. A transmission of this type is unique in that it cannot be reliably read or copied, and any attempts to do so are highly likely to be detected. It enables the establishment of correlated, partially secret data between two remote users, which is subsequently checked, error-free [4–6], and privacy amplified [7,8] through traditional methods that require a reliable, authenticated public channel. The nature of quantum transmission means that QKD is a distance-limited point-to-point technology [9–11]. To scale the technology to a multiuser, distance-independent secret key agreement, several networks of QKD links and nodes, i.e., QKD networks, have been accomplished [12]. The major challenge is to structure, organize, and manage this unique network to use both quantum and classic communications to establish ITS secret keys between arbitrary nodes.

However, the pace of progress is slow; even a small-scale QKD network is expensive and remains inaccessible to a large community of researchers. The need for accurate and scalable simulation technology to drive future development and teaching is therefore undeniable. While simulation tools designed for emerging quantum technologies are in abundance [13–18], none address the networking aspects of practical QKD networks and instead focus on entanglement-based quantum networks. These networks remain beyond the reach of current technology. Simulation technologies simulate critical quantum hardware and permit the development of software applications for these networks. These technologies will simplify the integration of such networks once the hardware is in place. QKD networks, however, have been in experimental testing for more than two decades, with commercial products already emerging and establishing themselves. These types of networks still have many challenges relating to the complex aspects of key and network management, efficient management of scarce key material, and providing reliable services with quality of service (QoS) assurance [19].

The paper therefore presents a vastly expanded model of the Quantum Key Distribution Network Simulation module

(QKDNetSim) [20]. It is set to become the essential tool for testing novel network management methodologies applied to QKD networks. The tool is capable of simulating large-scale QKD networks, i.e., networks with an arbitrary number of nodes, links, and consumers. The module was designed with existing practices and standards in mind, but it also offers new solutions and communication methods. The article provides more than just a description of the simulation tool; it offers valuable commentary on the tool's design and approaches to QKD network management and communications. As a complement to the article, we provide a user-friendly web adaptation of the module [21].

This paper is organized as follows. Section 2 presents the state of the art in QKD networks, with an emphasis on purpose, structure, and interfaces. Section 3 describes the developed simulation environment—QKDNetSim—and its major components. Two major components—a QKD cryptographic application and a QKD key manager system (KMS) application—are further detailed in Sections 4 and 5, respectively. Section 6 presents a verification of the proposed model. Section 7 briefly describes a user-friendly web adaptation of the developed module. Section 8 concludes the study.

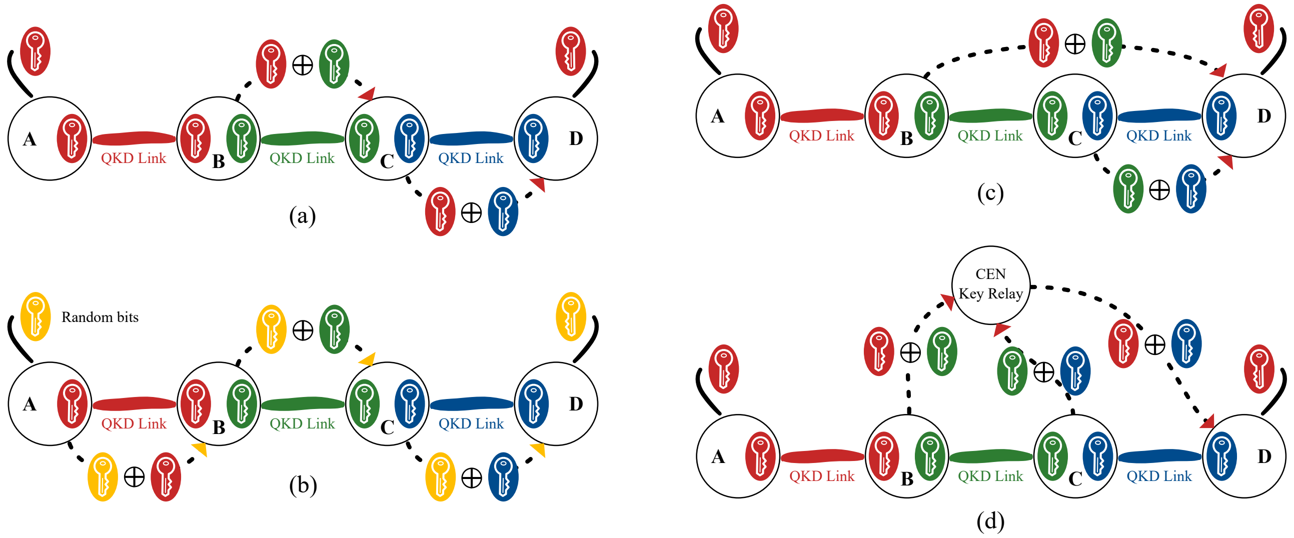
2. STATE OF THE ART

A. QKD Networks

To some extent, the limitations of standalone QKD technology can be overcome by constructing a network of individual QKD links [22,23]. The resulting network is called a QKD network. In the QKD network, arbitrary nodes establish ITS keys provided a chain of QKD links connects them. This fundamental requirement of the QKD network is known as key relaying [24] and may be accomplished in a variety of ways [25], as depicted in Fig. 1, but all require OTP encryption at each node in the connected chain, known as the relay path. This is the most common QKD network architecture in real-world applications, and it is referred to as a trusted-repeater QKD network because it requires all nodes in the network to be trusted. Selecting the best path, or route, is a fundamental requirement for a functional QKD network [12]. The tendency to keep the circle of trust as small as possible is consistent with the requirement to maintain low key network resource consumption. However, on some occasions, the shortest path is not the best option because the performance of the QKD process falls short of the required key relay's throughput across the given QKD link [26,27].

B. Layered Structure

The QKD network is defined as an add-on technology and service to conventional telecommunications networks [28,29]. The sole purpose of the QKD network is to generate, manage, and distribute cryptographic keys among arbitrary QKD nodes and to provide keys as a service to standard communications network users or applications [30]. The main objective and motivation is to increase communications security. The functional requirements [31] of the QKD network are divided into several distinct layers: the quantum layer, the key management layer, the QKD network control layer, the QKD network management layer, and the service layer [24,28], as illustrated in Fig. 2. The quantum layer is composed of pairs of QKD modules connected by logical QKD links that implement both the quantum and classic steps of the QKD process [28]. Key material generated by the quantum layer is accumulated at the key management layer, which is composed of key managers (KMs) and KM links. The key managers are in charge of key storage, key life cycle management, key relay, and key supply [25]. The KM link, which is a traditional network link, is used for key management functions. The security levels of KM links have rarely been studied, with the exception of key relay



(a)
(b)
(c)
(d)

Fig. 1. Different approaches to key relaying from node A to node D: (a) key relay from point to point (case 1), where the key shared between the source and the first hop is relayed to the destination node; (b) key relay from point to point (case 2), where the random key bits generated at the source node are relayed to the destination node; (c) key relay with XORs uniformly processed at the destination node, where all intermediate nodes send XOR values of the keys they share with their preceding and succeeding neighbors to the destination; and (d) key relay with XORs collected at a single, centralized node.

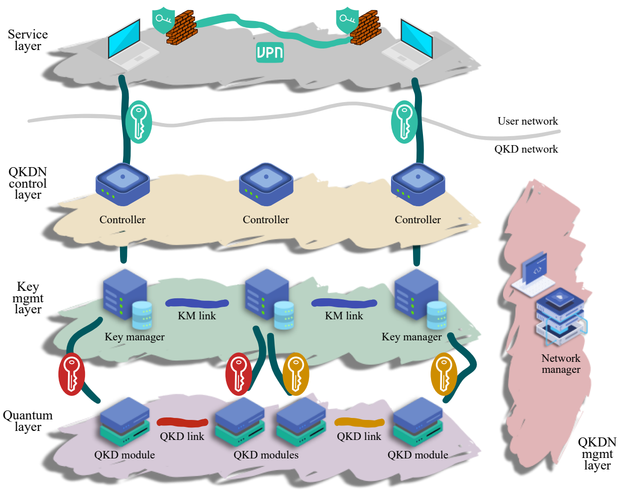

Service layer
VPN
User network
QKD network
QKDN control layer
Controller
Controller
Controller
Key mgmt layer
KM link
KM link
Key manager
Key manager
Quantum layer
QKD link
QKD link
QKD module
QKD modules
QKD module
Network manager
QKDN mgmt layer

Fig. 2. Layered structure of the QKD network based on ITU-T recommendations.

procedures that require OTP encryption. Quantum and key management layers are supported and controlled by the QKD network control layer, which implements control functions. The QKD network controller, or simply the controller, handles routing control, configuration control, policy-based control, and access and session control functions [32]. In the case of a centralized QKD network architecture, i.e., software-defined networking (SDN), a centralized controller supports a few additional functions [33] and interfaces [34,35]. The QKD network manager monitors and manages the entire QKD network, and a user layer contains cryptographic applications that employ the QKD network's services.

C. Key-Supply Interfaces

The key-supply interface describes access to the QKD network services by defining communications between cryptographic applications and KMs. ETSI has recently standardized two such interfaces: ETSI GS QKD 014 [36] and ETSI GS QKD 004 [37]. Through the ETSI specification, cryptographic applications are also referred to as secure application entities (SAEs), while KMs are referred to as key management entities (KMEs).

The ETSI QKD 014 specification defines an interface based on the HTTPS protocol and the JSON-encoded data format of posted parameters and responses. Because of its REST-based nature and the simplified processing logic required at the KMEs, this interface is well accepted among vendors who supply quantum equipment. The ETSI QKD 004 introduces the concept of sessions with QoS capabilities for SAEs without requiring the use of a specific protocol. The interface use cases are depicted in Figs. 3 and 4.

3. QKD NETWORK SIMULATION MODULE

Introduced in 2017, QKDNetSim [20] is the first computing simulation environment for QKD networks to focus on public channel performance and networking in the classic domain with a significantly simplified QKD process. The environment was developed in Network Simulator 3 (ns-3) as the best fit

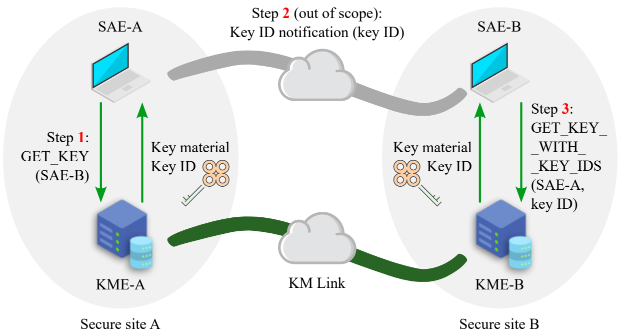

SAE-A
Step 2 (out of scope):
Key ID notification (key ID)
SAE-B
Step 1:
GET_KEY
(SAE-B)
Key material
Key ID
Key material
Key ID
Step 3:
GET_KEY_
WITH
_KEY_IDS
(SAE-A,
key ID)
KME-A
KM Link
KME-B
Secure site A
Secure site B

Fig. 3. Use case of the ETSI QKD 014 key-delivery interface.

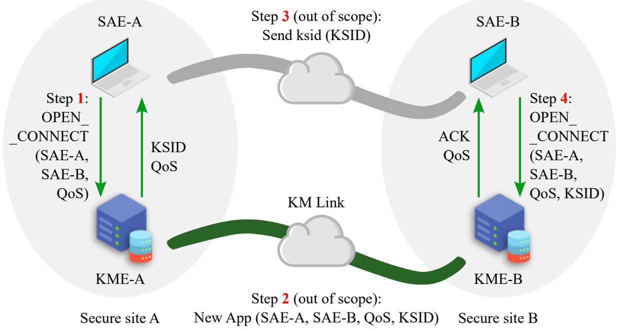

SAE-A
Step 3 (out of scope):
Send ksid (KSID)
SAE-B
Step 1:
OPEN_
_CONNECT
(SAE-A,
SAE-B,
QoS)
KSID
QoS
ACK
QoS
Step 4:
OPEN_
_CONNECT
(SAE-A,
SAE-B,
QoS, KSID)
KM Link
KME-A
KME-B
Step 2 (out of scope):
New App (SAE-A, SAE-B, QoS, KSID)
Secure site A
Secure site B

Fig. 4. Use case of the ETSI QKD 004 key-delivery interface in a single-link scenario. The diagram shows only the key stream establishment phase, after which SAEs obtain keys by issuing GET_KEY requests with the given KSID. Key reservation at the key management layer is also depicted; the key database is shown as a red segment.

for the following requirements: accuracy of results, extensibility, usability, and availability [20]. The original design of QKDNetSim offered a rather simplified view of the QKD network and its resources, with no key management and limited network management.

The article describes a new, significantly improved version of QKDNetSim (QKDNetSim is open-source, and the code is available at [38]) that incorporates the major functional requirements of QKD networks (Fig. 5). The development of QKDNetSim involved producing and implementing a range of methods not previously discussed in other research. For example, QKDNetSim defines a horizontal interface between peer KMs to perform key management tasks such as supply key creation and synchronization, key stream establishment, and key relaying across the network of KM links. These methods are detailed later in the text but are mentioned here to highlight the contribution of the research to developing new solutions for QKD networks in addition to the presented simulation environment.

A. Essential QKD Components and Architecture

This section describes the essential components, or classes, of the QKDNetSim simulation environment. Figure 6 shows their division into two groups—QKD core components and QKD applications—according to specific abstractions and purposes. Given the complexity of the latter group, the QKD KMS and QKD cryptographic applications are described here

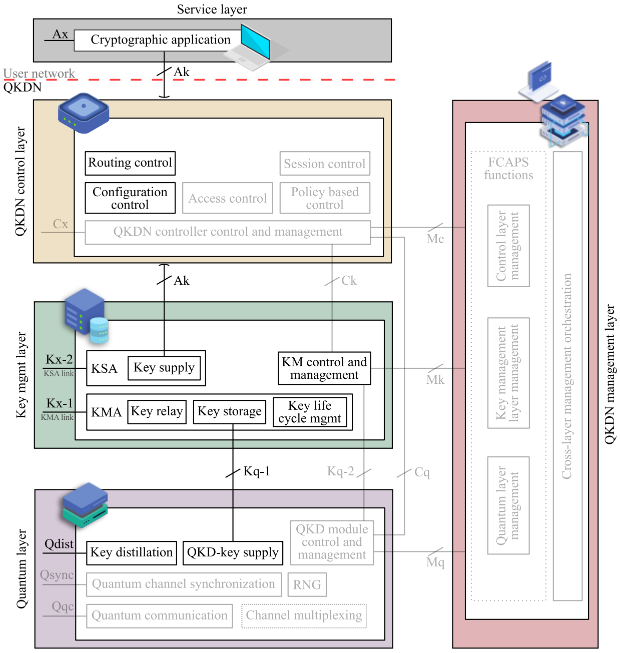

Service layer
Ax
Cryptographic application
User network
QKDN
Ak
QKDN control layer
Routing control
Session control
Configuration control
Access control
Policy based control
Cx
QKDN controller control and management
Mc
Ak
Ck
Key mgmt layer
Kx-2
KSA link
KSA
Key supply
KM control and management
Mk
Kx-1
KMA link
KMA
Key relay
Key storage
Key life cycle mgmt
Kq-1
Kq-2
Cq
Quantum layer
Qdist
Key distillation
QKD-key supply
QKD module control and management
Mq
Qsync
Quantum channel synchronization
RNG
Qqc
Quantum communication
Channel multiplexing
FCAPS functions
Control layer management
Key management layer management
Quantum layer management
Cross-layer management orchestration
QKDN management layer

Fig. 5. Implemented functions at each layer from the ITU-T functional architecture [24]. The faded functions and reference points are not yet supported. However, it should be noted that the remaining functions of the quantum layer will not be implemented given the intended use of the simulator.

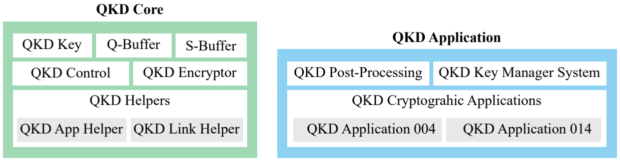

QKD Core
QKD Key
Q-Buffer
S-Buffer
QKD Control
QKD Encryptor
QKD Helpers
QKD App Helper
QKD Link Helper
QKD Application
QKD Post-Processing
QKD Key Manager System
QKD Cryptograhic Applications
QKD Application 004
QKD Application 014

Fig. 6. Essential components of the QKDNetSim ecosystem.

only briefly before being detailed in their respective sections. Figure 7 summarizes the function and relationship between these components along with their fundamental attributes.

1. QKD Key

A discussion of QKD networks is primarily about keys and their generation, storage, management, and use. In QKDNetSim, keys are pseudo-random character sequences generated by QKD post-processing applications. They are uniquely identified at the quantum layer using universally unique identifiers (UUIDs) [39,40] and 36 character strings and, at the key management layer, using eight character strings generated by a hash function (see Section 5.A). The truncated output of the hash function is sufficient to uniquely identify keys on a single QKD connection without collisions. In addition to unique identifiers, the keys are tagged with timestamps of their creation, length, and current state. The current state of the key assists the KMS in performing certain management tasks and can be set to INIT, READY, or OBSOLETE.

2. Q-Buffer

Due to the limited key generation rate of the QKD modules, keys are most commonly generated at the highest rate supported and then stored in a safe place for subsequent use in cryptographic processes. QKDNetSim therefore introduces the *Q*-buffer. Adjacent nodes with a logical QKD connection each implement one *Q*-buffer, as depicted in Fig. 8. *Q*-buffers have a limited capacity expressed in bits. Additionally, they have a minimum attribute that specifies the least amount of bits needed to start post-processing operations. The threshold value is defined, demonstrating that the required steps are taken before the buffer is entirely emptied. Constant monitoring of the state of the *Q*-buffer is therefore needed and assists with key management and key provisioning operations. Based on the quantity of secret bits retained and defined attributes, the *Q*-buffer may be in any of the following states:

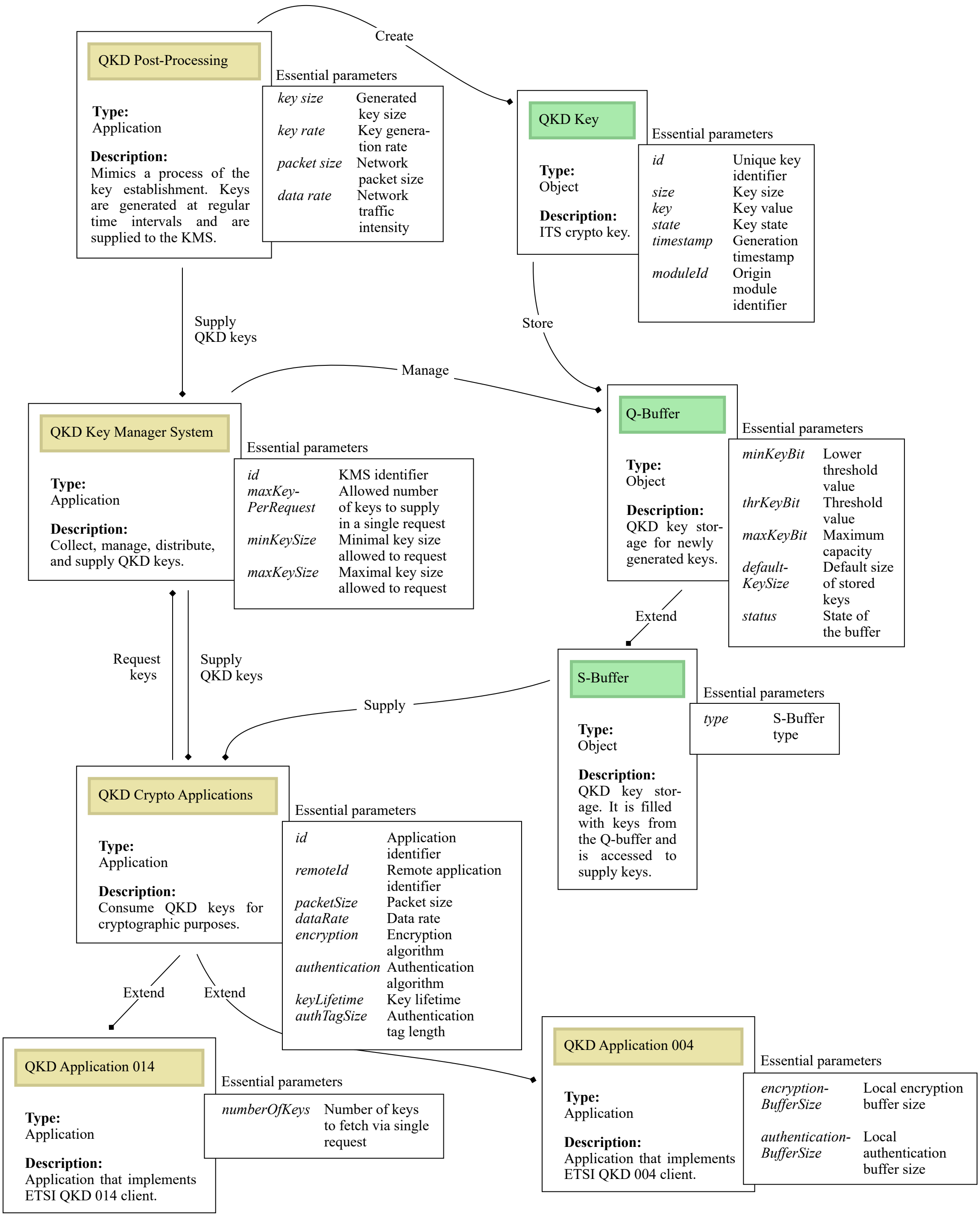
- READY: The amount of keys is greater than the threshold.
- WARNING: The amount of keys is less than the threshold, and the amount of keys in the buffer is decreasing or constant.
- CHARGING: The amount of keys is less than the threshold, and the amount of keys in the buffer is increasing.
- EMPTY: The amount of keys in the buffer is less than the minimum value.

Each *Q*-buffer has a standard size for the keys it stores. Before storing keys, the KMS is responsible for reformatting them into the correct size. This is accomplished using the key storage function described in Section 5.A. On occasion, keys are transferred from *Q*-buffers to *S*-buffers, which serve as working buffers.

3. S-Buffer

When this new, improved QKDNetSim environment was under development, keys were initially supplied from the *Q*-buffers. Without proper signaling at the key management layer, one KMS provided keys to a cryptographic application, while the peer KMS simultaneously used the same set of keys for different purposes. In this situation, the keys became useless, and any task they were required to complete would be suspended [41]. To avoid the shortcomings of the previous design, we employed the SECOQC–Q3P concept of separate key pools dedicated to outbound and inbound use [42]. QKDNetSim therefore introduces the concept of an *S*-buffer. The *S*-buffer class is inherited from the *Q*-buffer class and has functions and attributes in addition to those described in Section 3.A.2. The KMS initially creates a pair of *S*-buffers for each *Q*-buffer. These *S*-buffers are referred to as LOCAL and have two variants, *enc* and *dec*, as shown in Fig. 9. The KMS can use the *enc* *S*-buffer at any time, whereas the *dec* *S*-buffer is used only when instructed by the peer KMS.

As mentioned above, the *S*-buffers are filled with key material from the *Q*-buffers as necessary. For now, we assume that no reformatting operation is applied to keys as they transit from the *Q*-buffer to the *S*-buffer. However, we acknowledge the possibility of multiple pairs of LOCAL *S*-buffers being established based on key size criteria. Aside from the LOCAL



QKD Post-Processing

Type:
Application

Description:
Mimics a process of the key establishment. Keys are generated at regular time intervals and are supplied to the KMS.

Essential parameters

key size	Generated key size
key rate	Key generation rate
packet size	Network packet size
data rate	Network traffic intensity

QKD Key

Type:
Object

Description:
ITS crypto key.

Essential parameters

id	Unique key identifier
size	Key size
key	Key value
state	Key state
timestamp	Generation timestamp
moduleId	Origin module identifier

QKD Key Manager System

Type:
Application

Description:
Collect, manage, distribute, and supply QKD keys.

Essential parameters

id	KMS identifier
maxKeyPerRequest	Allowed number of keys to supply in a single request
minKeySize	Minimal key size allowed to request
maxKeySize	Maximal key size allowed to request

Q-Buffer

Type:
Object

Description:
QKD key storage for newly generated keys.

Essential parameters

minKeyBit	Lower threshold value
thrKeyBit	Threshold value
maxKeyBit	Maximum capacity
defaultKeySize	Default size of stored keys
status	State of the buffer

S-Buffer

Type:
Object

Description:
QKD key storage. It is filled with keys from the Q-buffer and is accessed to supply keys.

Essential parameters

type	S-Buffer type

QKD Crypto Applications

Type:
Application

Description:
Consume QKD keys for cryptographic purposes.

Essential parameters

id	Application identifier
remoteId	Remote application identifier
packetSize	Packet size
dataRate	Data rate
encryption	Encryption algorithm
authentication	Authentication algorithm
keyLifetime	Key lifetime
authTagSize	Authentication tag length

QKD Application 014

Type:
Application

Description:
Application that implements ETSI QKD 014 client.

Essential parameters

numberOfKeys	Number of keys to fetch via single request

QKD Application 004

Type:
Application

Description:
Application that implements ETSI QKD 004 client.

Essential parameters

encryptionBufferSize	Local encryption buffer size
authenticationBufferSize	Local authentication buffer size

Fig. 7. Attributes, relationships, and descriptions of QKDNetSim's essential components. The components QKD helpers, QKD encryptor, and QKD control are not displayed. Users can create QKD applications with the aid of QKD helpers. In addition to providing routing configuration, QKD control registers all QKD modules and cryptographic applications at their respective KMSs and configures the Q- and S-buffers. Cryptographic algorithms are executed by the encryptor component. To keep the illustration simple, only the most important features that might be of interest to the simulator's user are displayed.

S-buffers, the KMS can also establish RELAY and STREAM S-buffers, which are discussed in Sections 5.B and 5.C, respectively. The RELAY S-buffers, as the name implies, accumulate keys to a specific destination via the relay procedure, whereas the STREAM S-buffers represent reserved key streams in response to the ETSI QKD 004 clients.

4. QKD Control

The QKD control implements essential network controller functions. We refer to it simply as the *controller* in the rest of the article. The controller implements a subset only of the ITU-T functions defined for the control layer. The most

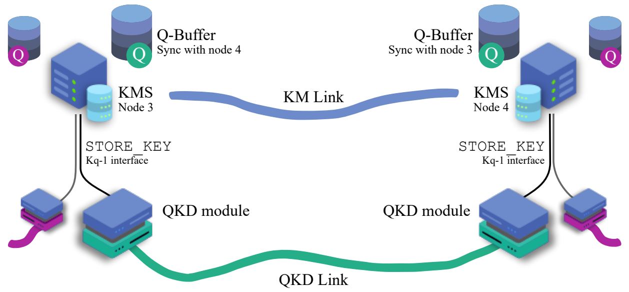


Fig. 8. Scheme of the Q-buffers.

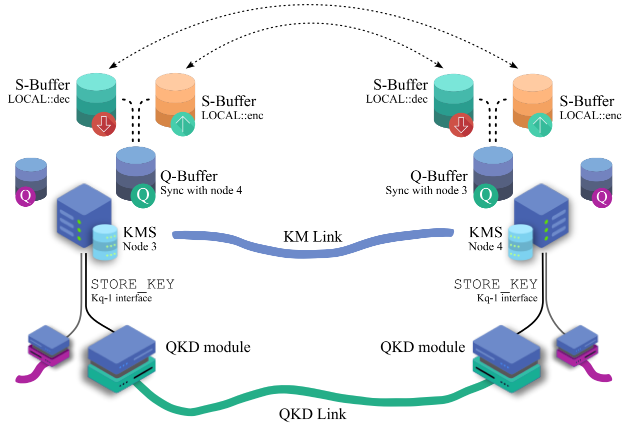


Fig. 9. Scheme of the S-buffers.

essential function, which allows simulation of large-scale networks, is the routing control. Controllers currently populate routing tables using Dijkstra's algorithm and detailed knowledge of the QKD network's topology. When the routing tables are calculated, the link weights are set to 1. Therefore, regardless of QKD link performance or key availability, the route with the fewest hops to the destination QKD node is selected. Clearly, routing requires further research and implementation of advanced algorithms that support QoS, but the current solution is a foundation that makes the work on large-scale networks possible and may ultimately simplify further development and modifications.

5. QKD Encryptor

The QKD encryptor is an entity that performs cryptographic operations such as encryption and authentication. It is implemented using Crypto++ open source cryptographic library [43]. It supports the use of the VMAC and SHA2 algorithms for authentication and OTP and AES for encryption. As cryptographic algorithms can place a significant computational load on the machines performing the simulation, QKDNetSim allows their execution to be switched off to enable efficient simulations of more significant QKD topologies.

6. QKD Helpers

QKDNetSim includes two helper classes, a QKD link helper and a QKD application helper, to simplify the installation of QKD entities at network nodes. The QKD link helper oversees the installation of controllers, configuration of buffer attributes for connections, collection of topology data, and creation of the topology graph. The QKD application helper oversees the installation of components assigned to the QKD applications group.

7. QKD Post-processing Application

Each of the numerous QKD protocols necessitates post-processing of the information obtained from the quantum transmission. Because the simulator is designed to simulate large-scale QKD networks and management methods rather than the process of establishing keys on the quantum channel (for which a large number of tools is listed in Section 1), communications on the quantum channel are ignored. The QKD post-processing application is therefore implemented to simulate a QKD protocol's network activity. This application only accurately mimics the traffic from already existing post-processing systems and does not bear any computational load from the execution of operations. This approach reduces the simulation time and conserves computational resources while enabling the effects to be studied of different parameters such as key generation rate, traffic volume, and network capacity on the state of the entire network. The QKD post-processing application has a number of configurable parameters that allow a reasonable adoption of many use cases. These parameters include average key rate, key size, network packet size exchanged during the post-processing phase, and traffic intensity. The QKD post-processing application is implemented according to a simple principle in which peers exchange counter values. The counter is reset regularly depending on the key rate setting, and packets are sent back and forth. Sifting packets are traded first when the counter is reset. At the end of the process, the generated keys are handed over to the KMS (see Section 5.A), where they are stored, managed, and consumed.

8. QKD Cryptographic Application

QKDNetSim implements two cryptographic applications, QKD Application 014 and QKD Application 004, to test QKD network performance and assign it purpose and meaning. These applications simulate QKD-enabled secure communications in various cryptographic or network organization settings and are thus consumers of QKD services. The primary distinction between the two applications is the ETSI interface used for communicating with the local KMS and, consequently, for internal key management logic at the application level. QKD cryptographic applications are detailed in Section 4.

9. QKD Key Manager System Application

The QKD Key Manager System Application, or the KMS, implements major functional requirements for the key management layer. It is a stand-alone software application that oversees key management and supply to cryptographic applications. Because the supply of QKD-derived key material is limited, how the KMS manages and provisions keys is critical.

To function correctly, the KMS must communicate with other components of the QKD network. For now, we assume that the KM links are secured using traditional methods that provide quantum-resistant authentication. The KMS is detailed in Section 5.

4. QKD CRYPTOGRAPHIC APPLICATIONS

The general purpose of the QKD cryptographic applications, or simply the *applications*, is to imitate the applications that consume cryptographic keys. It also evaluates the performance of the system as a whole under various conditions and illustrates which end-user services can be supported on the QKD network. Data communication is unidirectional, and the applications function in a master/slave paradigm. The master is the application that sends confidential data to the peer-slave, and the slave is a simple receiving application. Protected user data is encapsulated with a QKD Application Header and sent over a public, unsecured channel, as illustrated in Fig. 10. The QKD Application Header, shown in Fig. 11, contains key information (e.g., the applied cryptographic algorithms and the unique key identifiers of the cryptographic keys used for processing data) that assists the receiving applications in processing the protected (encrypted/authenticated) packets.

Communications between the applications and KMSs is generally established over the HTTPS protocol. QKDNetSim, however, primarily uses HTTP without TLS security. Although QKDNetSim can be upgraded with HTTPS support, it does not benefit from this for its intended purpose. (Implementing the HTTPS protocol in an ns-3 environment would require too much time and not contribute to the functionality of the simulator. The methods proposed in the HTTP variant are the same as those that would be used with HTTPS; the only difference is that HTTPS protocol messages are transmitted over a secure TLS connection at the transport layer. Because the primary goal of the proposed simulator is to provide a foundation for developing advanced key and network management methodologies in trusted-repeater QKD networks, implementation of the HTTPS protocol is not critical.) Any HTTP method defined in the article should therefore rely on HTTPS.

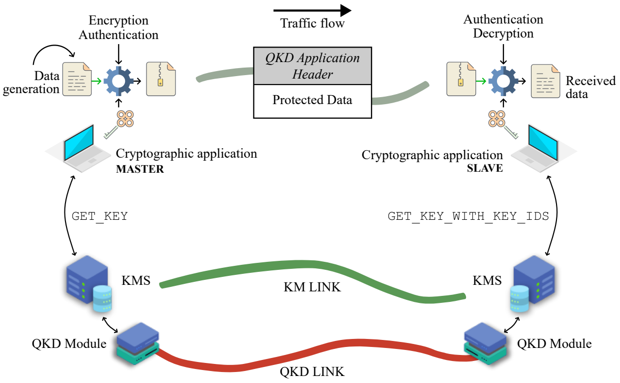

Encryption
Authentication
Traffic flow
Authentication
Decryption
Data generation
QKD Application Header
Protected Data
Received data
Cryptographic application
MASTER
Cryptographic application
SLAVE
GET_KEY
GET_KEY_WITH_KEY_IDS
KMS
KM LINK
KMS
QKD Module
QKD Module
QKD LINK

Fig. 10. General purpose of cryptographic applications. For illustration purposes, the diagram depicts only QKD Application 014.

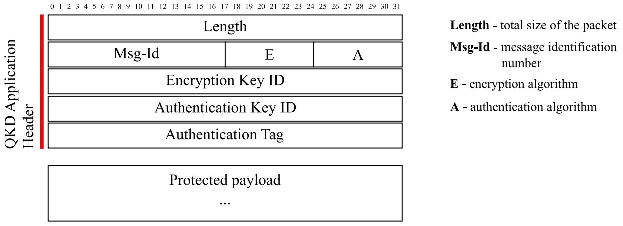

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
QKD Application Header
Length
Msg-Id
E
A
Encryption Key ID
Authentication Key ID
Authentication Tag
Protected payload
...
Length - total size of the packet
Msg-Id - message identification number
E - encryption algorithm
A - authentication algorithm

Fig. 11. QKD Application Header.

A. QKD Application 014

This section describes the implementation of QKD Application 014, or simply App014, which is a key consumption application for communicating with the local KMS via the ETSI QKD 014 interface. The application has numerous user-configurable parameters to support simulations of different use cases. Data rate, packet size, applied encryption and authentication protection, key lifetime, and the length of the authentication tag are examples of these parameters. App014 also lets users configure parameters related solely to the operation of this application; for example, users can specify the number of keys fetched by a single GET_KEY request or the time-out value that defines how long the application waits before issuing a new request if the previous request fails.

Because the system may obtain multiple keys, some key management logic must be applied at the application layer. QKDNetSim implements key stores at the application layer, and keys are sorted according to their use. Temporary and outbound key stores are specified on the sender's side and in the master App014, the latter being further subdivided into encryption and authentication stores, as depicted in Fig. 12. The receiver App014 implements the inbound key store only and applies a simpler key management logic. If both encryption and authentication algorithms require key material, both outbound encryption/authentication key stores must have valid entries before the application can securely transmit data. QKDNetSim implements a method mentioned in the ETSI interface standard on the service layer, i.e., key ID notification (Fig. 3). It is defined as the HTTP POST method–KEY_IDS

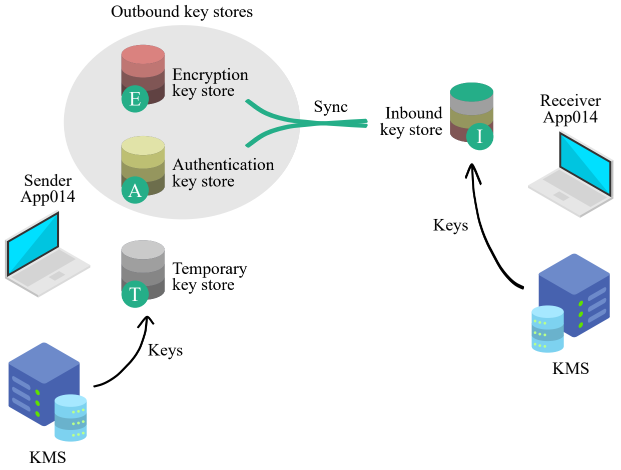

Outbound key stores
E
Encryption key store
A
Authentication key store
Sync
Inbound key store
I
Receiver App014
Sender App014
T
Temporary key store
Keys
Keys
KMS
KMS

Fig. 12. Scheme of key management at the application level. The sender application implements outbound and temporary key stores. Keys obtained from the KMS are stored in the temporary key store. After negotiation with the peer application, keys are moved to the appropriate outbound key store. The receiver stores all keys in the inbound key store.

with a request data model that corresponds to a key_IDs data format [36].

B. QKD Application 004

The QKD Application 004, or simply App004, serves the same purpose as App014 described in Section 4.A, but it communicates with the local KMS via an ETSI QKD 004 interface and therefore employs distinct key management logic. App004 maintains a key queue for each established key stream session. It also tracks the key streams, denoted by KSID, for both encryption and authentication. Before the application can begin secure communications, the queues must first accumulate the desired amount of key material. Once formed, the queues are synchronized by sending a new GET_KEY request to replenish the queue when a key is removed (i.e., the key's lifetime has expired). This design was inspired by the ideas presented in [44] for upgrading the IPsec protocol with QKD-derived keys [45].

To implement the ETSI QKD 004 interface, we selected the HTTP protocol and JSON data structures as obvious choices since we had already incorporated a variant of the HTTP protocol and combined it with JSON data formats. We used the same URI format as defined in the ETSI QKD 014 standard. On the application link, two additional methods are defined, CONNECT and ESTABLISH_QUEUES, the former of which addresses the necessary method mentioned in the standard itself, SEND_KSID (Fig. 4). The latter is applied to signal the beginning and end of the queue establishment phase, after which the application can begin secure communications. These methods are implemented using the HTTP protocol, and both apply the GET method to submit requests. The CONNECT request includes the KSID value and the scope of the session in the URI, whereas ESTABLISH_QUEUES includes no additional parameters, because it is presumed that the size of queues is known in advance by both peers. It is important to note that QKDNetSim currently does not permit the installation of App004 pairs on arbitrary nodes, only on adjacent nodes.

5. QKD KEY MANAGER SYSTEM APPLICATION

The KMS is an essential component of the QKD network's overall framework. It provides simple access to the QKD network's services for ITS key provisioning. Because the supply of keys is limited, the method of completing these tasks is critical. Producing this component demanded the majority of our work, particularly since methods at this layer are frequently overlooked or omitted in research papers. The article describes a custom implementation of the KMS and its methods for communicating with other components of the QKD network. We followed the ITU-T guidelines and applied standardized interfaces where possible (ETSI QKD 014 and ETSI QKD 004, in particular). The functionalities and interfaces of this custom KMS are described in the following sections.

A. Key Storage Function

This section describes a key storage function, which includes a description of the $K_q - 1$ interface (Fig. 6), a reformat function, and the FILL method specification.

1. STORE_KEY Method

First, we define the STORE_KEY method, which is specified for the $K_q - 1$ interface and used to transfer keys from QKD modules to the KMS. It is defined as an HTTP request submitted through the POST method. The parameters adhere to the ITU-T recommendation [25]. We added a QKD module ID and a matching QKD module ID, both in UUID format, to the list of mandatory parameters (which are optional in the ITU-T recommendations). The QKD module ID is used to identify the Q-buffer where keys will be stored. In conjunction with the matching QKD module ID, the KMS pair can then generate a unique set of IDs for reformatted keys. Finally, the STORE_KEY body contains the QKD key value in string format. (In practice, the Base64 encoded value may also be used.)

2. Reformat Function

This novel KMS design assumes that for any pair of connecting KMSs (each housing a paired QKD module), a KMS with a higher node ID in the simulation is selected as the master. In practice, this can be accomplished by comparing KMS unique identifiers or IPv4 addresses. The QKD key received from the layer below is split by the KMS into blocks of defined size, and the unique IDs generated must be synchronized with the remote KMS. The master KMS generates IDs in the following manner:

$$\begin{aligned} \mathrm{HASH-SHA} \quad & (\mathrm{QKD-key\ ID}\,| \\ & \mathrm{QKD\ module\ ID}\,| \\ & \mathrm{matching\ QKD\ module\ ID}\,| \\ & \mathrm{chunk\ number}), \end{aligned}$$

where the chunk number is a sequence number of the key block. To generate the same IDs, the remote KMS (which is not the master), only switches the position of QKD module ID and matching QKD module ID values during the concentration process. A hash function's output is truncated to eight character strings. We currently use SHA1 as a hash function, but we are examining other options that work well with short inputs and produce the shortest outputs with a low probability of collisions. Although SHA1 does not represent a security-acceptable solution, it is appropriate for generating unique IDs (which do not have to be kept secret) based on nonsecret inputs.

3. FILL Method

We apply the FILL method at the KM link to pour keys from the Q-buffer into the LOCAL and STREAM S-buffers. It is an HTTP request submitted through the POST method to enable the master KMS to instruct the slave KMS on which keys to transfer and to which S-buffer type. The body of this message therefore includes a source node ID, the key IDs array of objects, and the S-buffer type. The available keys from the Q-buffer are distributed evenly to the *enc* and *dec* LOCAL S-buffers. The STREAM S-buffer is always filled with up to six times the size of the key chunk. When the KMS sends a request to fill the STREAM S-buffer, it includes the KSID value. The response includes a key IDs array of objects containing the keys accepted from the proposal.

4. Key Validation

The KMS is required to synchronize and validate the keys obtained from QKD modules, for example, by exchanging hash values. The key material verification procedure is not currently implemented in the KMS of QKDNetSim. Reformatted keys are simply stored in a particular Q-buffer. Keys are tagged as READY. QKD post-processing applications must therefore be installed so that a master application is always connected to the slave KMS. By supplying keys first to the slave KMS, we avoid situations when the master KMS attempts to use key material that does not exist in the slave KMS. This procedure can be implemented according to the STORE subprotocol of the Q3P design [42].

B. Key Relay Function

The novel KMS design supports a key relay function across trusted-repeater QKD nodes in large-scale QKD networks. The key relay procedure is illustrated in Fig. 13 and corresponds to case (a) shown in Fig. 1. Whenever a cryptographic application requests a connection with a peer located at a distant QKD node (step 1), the KMS creates a RELAY S-buffer for this destination if one does not already exist (step 3). Before this, the KMS communicates with the controller to determine the destination SAE location, i.e., the destination KMS, and the next hop along the relay path (step 2). Then, it selects keys from the LOCAL *enc* S-buffer and moves them to the RELAY S-buffer, tagging them as INIT (step 4). At this stage, stored keys are not yet usable until they are confirmed by the destination KMS after successful relaying. At the key management layer, the RELAY request is sent to the next hop towards the destination (step 5); the data request format for the HTTP request at this step is shown in Table 1. The KMS at the first hop obtains keys from the LOCAL *dec* S-buffer (step 6), identified by the received key IDs from the RELAY request, and encrypts them with keys from the LOCAL *enc* S-buffer shared with the next hop (step 8) (before encryption, routing information is obtained from the controller—step 7). To make this procedure seamless, it is assumed that the Q-buffers, and therefore also the S-buffers, hold keys with the same default size for each link in the QKD network. Continuing with the key relay, each receiving node along the path uses encryption key

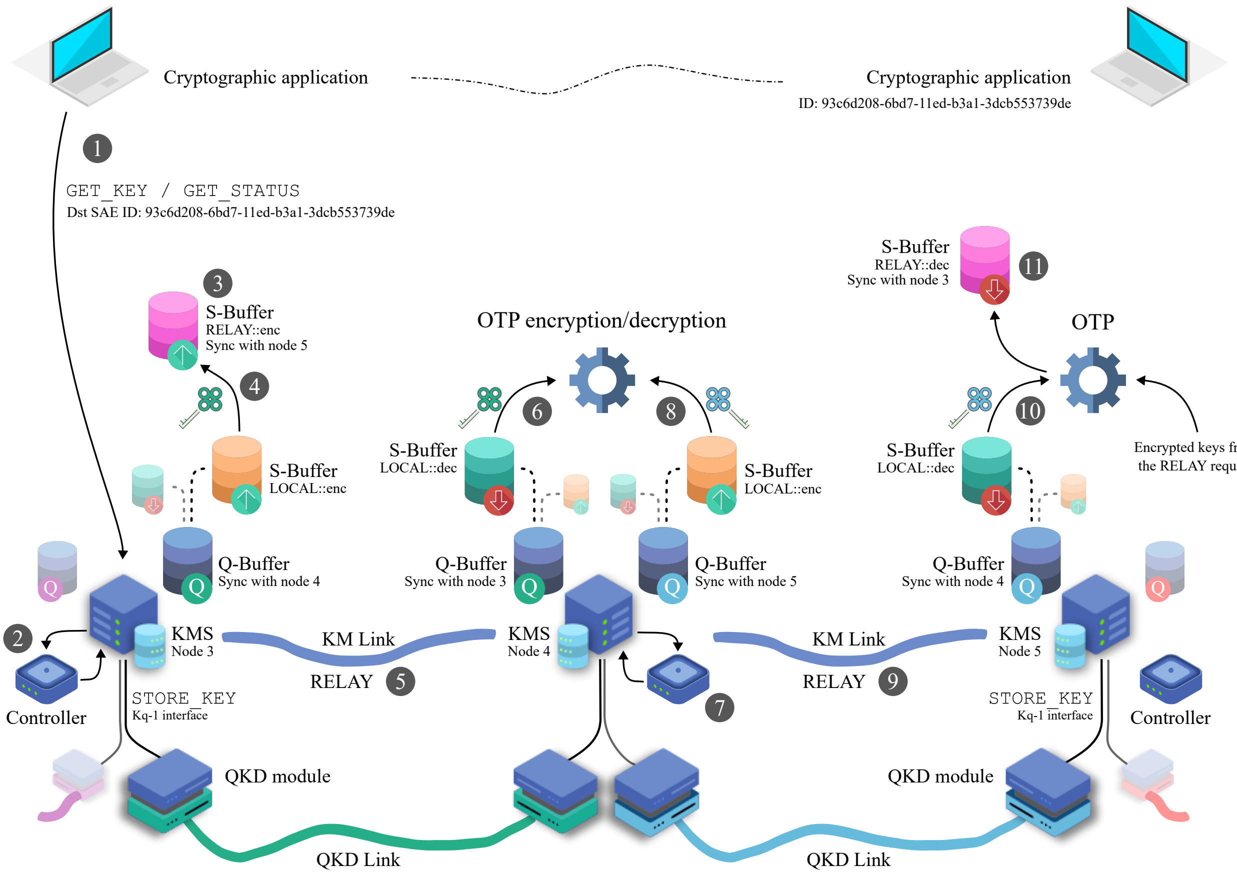

Cryptographic application
Cryptographic application
ID: 93c6d208-6bd7-11ed-b3a1-3dcb553739de
1
GET_KEY / GET_STATUS
Dst SAE ID: 93c6d208-6bd7-11ed-b3a1-3dcb553739de
3
S-Buffer
RELAY::enc
Sync with node 5
4
OTP encryption/decryption
6
8
S-Buffer
RELAY::dec
Sync with node 3
11
OTP
10
S-Buffer
LOCAL::enc
S-Buffer
LOCAL::dec
S-Buffer
LOCAL::enc
S-Buffer
LOCAL::dec
Encrypted keys from the RELAY request
Q-Buffer
Sync with node 4
Q-Buffer
Sync with node 3
Q-Buffer
Sync with node 5
Q-Buffer
Sync with node 4
Q
Q
Q
Q
Q
Q
2
KMS
Node 3
KM Link
RELAY
5
KMS
Node 4
7
KM Link
RELAY
9
KMS
Node 5
Controller
STORE_KEY
Kq-1 interface
STORE_KEY
Kq-1 interface
Controller
QKD module
QKD module
QKD Link
QKD Link

Fig. 13. Key relay procedure.

Table 1. Request Data Model for the RELAY Method

Items		Data Type	Description
source_node_id		uint32_t	Source KMS node ID.
destination_node_id		uint32_t	Destination KMS node ID.
repeater_node_id		uint32_t	Previous hop along the path identified with the KMS node ID. The initiator, i.e., the source KMS, does not include this parameter in the RELAY request.
keys		array of objects	A set of key identifiers and keys to relay.
	key_ID	string	ID of the key: UUID format. This is the ID of a key moved from the *S*-buffer *enc* to the RELAY *S*-buffer at the source node.
	ekey_ID	string	ID of the encryption key: UUID format. The encryption key used in the relay nodes with OTP encryption. The source node sends the key RELAY request without this parameter.
	ekey	string	Encrypted key. The encryption key is identified with the ekey_ID parameter.

IDs (*eKey_ID* within the RELAY request—step 9) to obtain the decryption keys from the LOCAL *dec S*-buffer shared with the previous node, decrypt the relay keys, access the LOCAL *enc S*-buffer shared with the next hop, and encrypt the keys to the next hop. This procedure is repeated for each hop along the relay path until the request reaches the destination node. (The illustrated key relay example omits an intermediate node between the first hop and the destination. To provide a full description of the relay procedure, we describe what could happen in step 9 if there are additional intermediate nodes. This process is not depicted in the illustration, however.) The destination node decrypts the relay keys (step 10), creates the RELAY *S*-buffer if it does not exist, and stores the keys as READY (step 11). The destination node now sends a reverse response to the source node, indicating a successful key relay. Because QKDNetSim uses HTTP as a protocol of choice to implement the interface between the KMSs, the RELAY method is implemented through the chain of responsibility pattern, or HTTP chain of proxies. Each hop modifies the previous request's content before passing it on. The response also travels in a hop-by-hop manner in the opposite direction until it reaches the source. Once the response reaches the source, the source KMS tags the keys as READY and is able to serve ETSI QKD 014 requests for the distant location. The source KMS monitors the state of the RELAY *S*-buffer, and new RELAY requests are issued when the available key material in the *S*-buffer falls below the threshold value. If the relay procedure fails at any of the intermediate nodes, the relay keys at that node are discarded, and an error response is returned to the source node. Likewise, the source node discards a specified set of keys.

C. Key Supply Function

The key supply function provides access to QKD network services by implementing two API clients: ETSI QKD 014 and ETSI QKD 004. This section describes the functional element's responsibilities and capabilities during steps taken on given API methods.

1. ETSI QKD 014 GET_STATUS

The GET_STATUS method provides cryptographic applications with information about the state of the underlying QKD service [36]. Depending on the desired cryptographic application to be reached, the KMS provides status information in relation to the state of the particular *S*-buffer, either LOCAL or RELAY. The KMS assumes that the status information for the outbound keys is requested and that the state of the *S*-buffers serving the purpose of *enc* is examined. The other parameter, i.e., the target_KME_ID information, is requested from the controller that supplies the routing information.

2. ETSI QKD 014 GET_KEY

When a GET_KEY request is received, the request parameters are first validated. Several error messages, whose data format is specified by the ETSI QKD 014 standard, are implemented. Let us suppose that the validation process is successful, which means that the parameters adhere to the rules of the KMS and that the target *S*-buffer (LOCAL *enc* or RELAY *enc*) holds sufficient keys to satisfy the request. In such a case, the KMS obtains the requested number of keys from the *S*-buffer and provisions them to the cryptographic application. The submitted keys are then removed from the *S*-buffer.

Cryptographic keys are generally supplied according to key size criteria, and keys must be reformatted or transformed to satisfy the request. Furthermore, because the keys are not identified with UUIDs at the key management layer, even if the requested key size corresponds to the default key size, new IDs must be assigned to the keys to comply with the standard. We propose the HTTP method SKEY_CREATE to fulfill these two requirements. Supply keys are created in the following manner. The KMS obtains key material in an amount equal to the total requested amount (number of keys * key size). Then, the KMS divides the merged value of the keys into the desired block size and assigns a UUID identifier to each. Consequently, the keys can be provided to the requesting application. Nonetheless, the KMS must notify the peer KMS of the creation and supply of these keys since the same set will be requested at the other end. It therefore sends the

SKEY_CREATE request, whose body includes a set of supply IDs (in UUID format) and a complete, ordered set of keys, i.e., candidate key IDs, used in the transformation process. The request also includes the source node ID, target SAE ID, size, and number of keys. After receiving the request to create the supply keys, the destination KMS checks the availability of the keys identified with the IDs carried in the candidate set ID array. If the keys are present in the S-buffer *dec*, the KMS creates supply keys and stores them in the dedicated storage (for supply keys only) of the corresponding S-buffer.

3. ETSI QKD 014 GET_KEY_WITH_KEY_IDS

It is easy to conclude from the previous section and description of the SKEY_CREATE method that the GET_KEY_WITH_KEY_IDS procedure is simple: the request submits a set of key IDs that the cryptographic application (SAE) wants to obtain; the keys are located in the section of the S-buffer (LOCAL or STREAM) for decryption (*dec*); the KMS verifies that the keys are intended for the requesting SAE; after providing the desired set of keys, the keys are removed from the S-buffer.

4. ETSI QKD 004 OPEN_CONNECT

The cryptographic application sends the OPEN_CONNECT request to establish a dedicated key stream session with guaranteed QoS criteria. However, the current version of the KMS does not implement QoS logic; it instead allows the creation of new sessions with no QoS guarantees. The KMS differentiates between the sender's and the receiver's cryptographic application calls based on the presence of the KSID parameter. This follows from the current implementation of the KMS, which does not support the predefined KSID scenarios described in the ETSI 004 standard.

The OPEN_CONNECT request (with the valid data formats and parameters) submitted by the sender application results in establishing a new key stream session. The KMS generates a unique key stream identifier, i.e., KSID, stores the session details, and sets a dedicated STREAM S-buffer. Since QKDNetSim supports only single-link scenarios for the application that rely on ETSI QKD 004, the response on the OPEN_CONNECT call is sent immediately, without any rendezvous with the peer or an intermediate KMS. The response carries the KSID for the granted session. In addition, the KMS sends a NEW_APP message to its peer KMS to advertise the establishment of the new session. The request data model for this HTTP request includes the source SAE ID, destination SAE ID, QoS structure, and KSID value.

The OPEN_CONNECT request submitted by the receiver application requires basic validation, and the KMS must assure that the KSID is registered for the application that made the request. If the validation fails, the KMS responds with an error message. Otherwise, the KMS acknowledges the registration, sends a response on the OPEN_CONNECT call, and informs the peer KMS through a REGISTER notification. The sole purpose of the REGISTER notification is to announce when the receiver application connects to the previously advertised key stream session. This is an important detail revealed by the standard in describing errors in the GET_KEY request. The sender App004 should not be able to obtain the key material until its peer registers for the same session to the corresponding KMS.

5. ETSI QKD 004 GET_KEY

An authorized cryptographic application submits a new key request through the GET_KEY method. The KMS checks the KSID included in the URI of the incoming request. If the assigned session STREAM S-buffer is not empty, it provides a successful response, which includes Index and Key_buffer values (see the ETSI QKD 004 specification [37]). Otherwise, it responds with an error. Implementing GET_KEY clearly shows that the KMS does not support requests submitted for a specific Index; instead, the applications are bound to obtain keys in an ordered manner. This current implementation design is important for the CLOSE_KMS method on the KMS, described in Section 5.C.6. When the KMS provides a new key to the sender application, it checks the state of the session buffer and, if necessary, starts a new FILL procedure.

6. ETSI QKD 004 CLOSE

After receiving the CLOSE request, the KMS initiates the CLOSE_KMS procedure at the key management layer. It indicates success, however, to the application before the CLOSE_KMS procedure is complete. The KMS assures that the key material in the STREAM S-buffer is preserved and that the session entry is removed. Currently, the KMS does not hold any information about the previously closed session.

The purpose of the CLOSE_KMS method (named to differentiate it from the CLOSE method defined by the ETSI 004 standard) is to synchronize the closure of key stream sessions between the KMSs. The most important task is to preserve the remaining key material in the STREAM S-buffer, but in a synchronized manner. This procedure is explained below:

- The KMS, which receives the CLOSE request, initiates CLOSE_KMS. It defines a key ID (the surplus key ID) to derive a number of IDs for the key material present in the STREAM S-buffer. In addition, it states the first index (i.e., the sync index) of the key present in the STREAM S-buffer.
- The peer KMS reads the sync index from the request and defines its own based on its STREAM S-buffer state. The KMS combines keys with indices greater than two: the sync index and the local sync index. It also responds to a CLOSE_KMS request and states its local sync index so that initiating the KMS can also trigger the key preservation procedure.
- At both KMSs, the merged key is first divided into blocks of the default size for a given Q-buffer. Each block is assigned a new ID. The IDs are generated in a manner similar to the procedure described in Section 5.A. The inputs to the hash function are a concentration of surplus key ID, KSID value, and a block index. The keys are then stored in the Q-buffer.

6. VERIFICATION OF THE MODEL

In this section, we verify the simulator by simulating a QKD network with six nodes and three user traffic flows. The results were examined to ensure that the simulator functions properly. The network topology and link performance are shown in Fig. 14. All links generate keys in time intervals ranging from 0 to 100 s. Links 1–2 and 2–3 correspond to a free-space QKD with limited reach and performance. Classic network links have a throughput of 100 Mbps and delay of 2 ms. Table 2 summarizes the cryptographic application settings.

Table 3 shows the key generation versus consumption on each QKD link. The set key rate is matched by the total number of keys generated on QKD links in 100 s. Because link 5–6 is not in use, it does not provide any services to users or relay any keys on their behalf. Links 1–2 and 2–3 are relaying keys for all users on demand, while links 3–4 and 4–5 are on a relay path for users 1–5 and 5–1, and link 3–6 is on a relay path for user 1–6. (User x-y is a pair of cryptographic applications located on sites x and y, with the master/sender application on node x.) These paths are the shortest paths between corresponding users calculated according to Dijkstra's algorithm. The limited key generation rate on the free-space channels and the user demands resulted in links 1–2 and 2–3 providing a key relay throughput near peak performance. Because link 2–3 relayed more keys than link 1–2, it indicates that the key relay failed a number of times, wasting key material. This is due to the lack of key material on link 1–2 for relaying keys on behalf of user 5–1. The table shows the amount of key material served to the cryptographic applications. Because all the connections are virtual and established via trusted relays, the key material to serve is derived from the relayed key material. In this sense, the amount of key material served is always less than or equal to the amount of relayed keys. This amount corresponds to the sum of the cryptographic keys received by the users (results in Table 4). Two OTP secured traffic flows, 1–5 and 5–1, run for the same time (60 s) but at different data rates (see Table 2). As a result, the higher data rate flow consumes more keys in total. This consumption difference should ideally be doubled, but flow 5–1 experienced service degradation. As mentioned above, link 1–2 was unable to meet the demands for relay throughput, resulting in a lack of key material for user 5–1, as indicated by missing send data calls. Furthermore, in the case of these two OTP flows, the number of consumed keys should be equal to the number of sent packets. However, the results (Table 4) show that this is not always the case. In this simulation experiment, the number of keys to retrieve from the KMS via a single request is set to three. The application consequently recorded this consumption but was unable to use the internally managed key material because of the stop time. Interruptions in data flow 1–6 are to be expected because of the key management logic implemented at the application level and the monitoring of missing send packet calls. Initially, user 1–6 expects to begin a secure data flow, but the relay S-buffers for this virtual connection have not yet been established at the key management layer. The relay procedure may take some time, but the user tracks the missing send packet calls, i.e., the instances when it expects to send packets of a certain size in order to achieve the desired data rate. In addition, the user does not issue new key requests until all the keying material has been used at the application layer. Consequently, because of high data rates, the key material may not be provided in such a short period, causing an interruption in secure data flow.

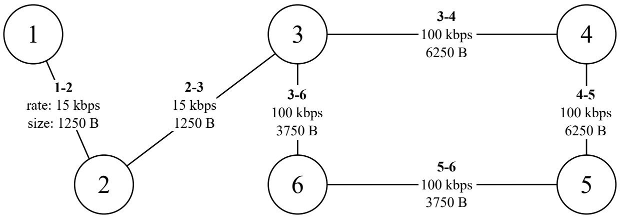


Fig. 14. Padua QKD network [46] topology as simulated in the QKDNetSim environment. It consists of six nodes and six links (1–2 and 2–3), two being distance and performance limited and representing a free-space QKD, the other four representing fiber-based versions of a QKD with a key rate of 100 kbps. Consumer application pairs are installed on nodes 1–5, 5–1, and 1–6, with traffic flowing from the first node to the second in each pair.

Table 2. Cryptographic Application Settings

Cryptographic Application	Interface	Data Rate (kbps)	Packet Size (bytes)	Encryption	Authentication	Key Lifetime (bytes)	Start Time (s)	Stop Time (s)
1–5	014	7	300	OTP	SHA2	/	15	75
5–1	014	14	300	OTP	SHA2	/	25	85
1–6	014	10.000	800	AES-256	SHA2	300,000	10	130

Table 3. Link Statistics

Link	Set Key Rate (bps)	Gen. Running Time (s)	Avg. Gen. Key Size (bits)	Generated Keys	Generated Keys (bits)	Achieved Key Rate (bps)	Relayed Keys	Relayed Keys (bits)	Served Keys (bits)
1–2	15.000	100	10.000	149	1.490.000	14.900	2.348	1.202.176	1.123.680
2–3	15.000	100	10.000	149	1.490.000	14.900	2.390	1.223.680	0
3–4	100.000	100	50.000	199	9.950.000	99.500	2.280	1.167.360	0
4–5	100.000	100	50.000	199	9.950.000	99.500	2.280	1.167.360	1.000.800
5–6	100.000	100	30.000	333	9.990.000	99.900	0	0	0
3–6	100.000	100	30.000	333	9.990.000	99.900	290	148.480	122.880

Table 4. User Statistics

Cryptographic Application	Sent (bytes)	Received (bytes)	Sent (packets)	Received (packets)	Missed Send Data Calls	Keys Consumed	Keys Consumed (bits)
1–5	67,396	67,396	166	166	9	168	403,200
5–1	101,094	101,094	249	249	101	249	597,600
1–6	162,810,918	162,807,294	179,703	179,699	7779	480	122,880

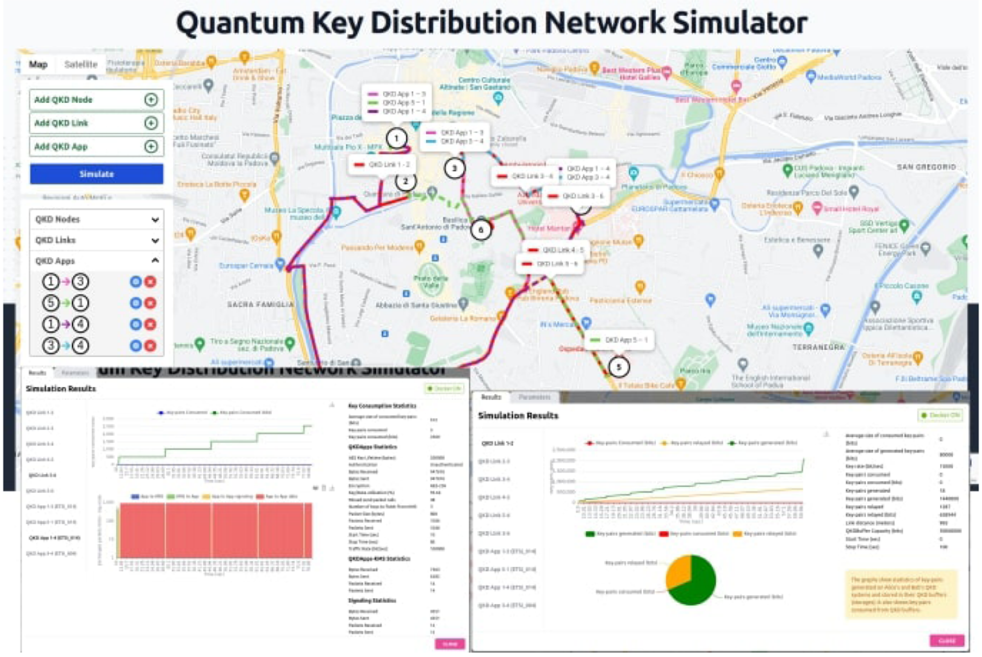


Fig. 15. Users can work with the QKDNetSim web interface at [47] to select map locations associated with QKD systems, KMS entities, and QKD applications (ETSI 004 and ETSI 014). The simulations provide statistics on the number of keys produced and consumed, the number of keys processed on the KMS, and, based on the input data, the amount of traffic exchanged with the KMS and between apps.

Based on the presented results, we conclude that the developed functional elements operate correctly and that the simulator produces reliable simulations of QKD networks.

7. QKDNetSim WEB INTERFACE

To improve the accessibility to QKD network simulations, a publicly accessible web interface has been developed and is available at [21,47]. The QKDNetSim web interface is intuitive and easily configured for a variety of experiments. The configuration parameters of the available components provide great flexibility for experimentation and the creation of unique scenarios. Some advanced parameters, however, are hidden to avoid obscuring the user interface, and others are limited to a set of available values. The simulation results page includes a large number of traced key network performance indicators, a graphical representation of QKD buffer states in time, and application-specific graphs for key consumption. A screenshot of the web interface, including the experimental setup and results page, is shown in Fig. 15.

8. CONCLUSION

The article describes an extended model of QKDNetSim, which was developed with the aim of being an indispensable tool for testing novel network management methodologies applied to large-scale QKD networks. The model includes an implementation of major QKD network components such as cryptographic applications, key managers, and QKD modules. It closely resembles the structures of contemporary QKD networks and supports two major key delivery interfaces standardized by ETSI, ETSI QKD 014, and ETSI QKD004. The article also proposes communication methods at the application and key manager links not previously covered in standards or other research papers. The simulation results demonstrate that functional elements and interfaces behave correctly, and that the simulator is accurate and practical. The open web interface enables a simple introduction to the simulator and flexible simulations of various use cases.

Funding. Ministry of the Interior of the Czech Republic (VJ01010008); NATO SPS (G5894); Horizon 2020 Framework Programme (857156); Ministry of Science, Higher Education and Youth of Canton Sarajevo, Bosnia and Herzegovina (27-02-35-37082-1/23).

REFERENCES

1. C. H. Bennett and G. Brassard, “Quantum cryptography: public key distribution and coin tossing,” in *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing* (1984), Vol. **175**, p. 8.

2. G. S. Vernam, "Cipher printing telegraph systems: for secret wire and radio telegraphic communications," J. AIEE **45**, 109–115 (1926).
3. C. E. Shannon, "Communication theory of secrecy systems," Bell Syst. Tech. J. **28**, 656–715 (1949).
4. C. H. Bennett, F. Bessette, G. Brassard, *et al.*, "Experimental quantum cryptography," J. Cryptol. **5**, 3–28 (1992).
5. G. Brassard and L. Salvail, "Secret-key reconciliation by public discussion," in *Workshop on the Theory and Application of of Cryptographic Techniques* (Springer, 1993), pp. 410–423.
6. M. Mehic, M. Niemiec, H. Siljak, *et al.*, "Error reconciliation in quantum key distribution protocols," in *Reversible Computation: Extending Horizons of Computing* (2020), p. 222.
7. C. H. Bennett, G. Brassard, and J.-M. Robert, "How to reduce your enemy's information," in *Advances in Cryptology—CRYPTO'85 Proceedings 5* (Springer, 1986), pp. 468–476.
8. C. H. Bennett, G. Brassard, and J.-M. Robert, "Privacy amplification by public discussion," SIAM J. Comput. **17**, 210–229 (1988).
9. M. Lucamarini, Z. L. Yuan, J. F. Dynes, *et al.*, "Overcoming the rate–distance limit of quantum key distribution without quantum repeaters," Nature **557**, 400–403 (2018).
10. Z. Yuan, A. Plews, R. Takahashi, *et al.*, "10-Mb/s quantum key distribution," J. Lightwave Technol. **36**, 3427–3433 (2018).
11. Y. Zhang, Z. Chen, S. Pirandola, *et al.*, "Long-distance continuous-variable quantum key distribution over 202.81 km of fiber," Phys. Rev. Lett. **125**, 010502 (2020).
12. M. Mehic, M. Niemiec, S. Rass, *et al.*, "Quantum key distribution: a networking perspective," ACM Comput. Surv. **53**, 96 (2020).
13. A. Dahlberg and S. Wehner, "SimulaQron—a simulator for developing quantum internet software," Quantum Sci. Technol. **4**, 015001 (2018).
14. T. Coopmans, R. Knegjens, A. Dahlberg, *et al.*, "NetSquid, a network simulator for quantum information using discrete events," Commun. Phys. **4**, 164 (2021).
15. X. Wu, A. Kolar, J. Chung, *et al.*, "SeQUeNCe: a customizable discrete-event simulator of quantum networks," Quantum Sci. Technol. **6**, 045027 (2021).
16. R. Satoh, M. Hajdušek, N. Benchasattabuse, *et al.*, "QuISP: a Quantum Internet Simulation Package," arXiv, arXiv:2112.07093 (2021).
17. S. DiAdamo, J. Nötzel, B. Zanger, *et al.*, "QuNetSim: a software framework for quantum networks," IEEE Trans. Quantum Eng. **2**, 2502512 (2021).
18. B. Bartlett, "A distributed simulation framework for quantum networks and channels," arXiv, arXiv:1808.07047 (2018).
19. M. Mehic, S. Rass, P. Fazio, *et al.*, "Modern trends in quantum key distribution networks," in *Quantum Key Distribution Networks: A Quality of Service Perspective* (2022), pp. 209–223.
20. M. Mehic, O. Maurhart, S. Rass, *et al.*, "Implementation of quantum key distribution network simulation module in the network simulator NS-3," Quantum Inf. Process. **16**, 253 (2017).
21. F. Kutschera, E. Dervisevic, L. Behan, *et al.*, "Data acquisition and simulation tools for virtual QKD testbed access—examples from the OPENQD project," in *European Conference on Optical Communication (ECOC)* (IEEE, 2021).
22. C. Elliott, "Building the quantum network," New J. Phys. **4**, 46 (2002).
23. C. Elliott, D. Pearson, and G. Troxel, "Quantum cryptography in practice," in *Proceedings of the 2003 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications* (2003), pp. 227–238.
24. "Quantum key distribution networks—Functional architecture," ITU-T Recommendation Y.3802 (2020).
25. "Quantum key distribution networks—Key management," ITU-T Recommendation Y.3803 (2020).
26. Y. Tanizawa, R. Takahashi, and A. R. Dixon, "A routing method designed for a quantum key distribution network," in *8th International Conference on Ubiquitous and Future Networks (ICUFN)* (IEEE, 2016), pp. 208–214.
27. C. Yang, H. Zhang, and J. Su, "The QKD network: model and routing scheme," J. Mod. Opt. **64**, 2350–2362 (2017).
28. "Overview on networks supporting quantum key distribution," ITU-T Recommendation Y.3800 (2019), Corrigendum 1 (2020).
29. M. Mehic, L. Michalek, E. Dervisevic, *et al.*, "Quantum cryptography in 5G networks: a comprehensive overview," IEEE Commun. Surv. Tutorials **26**, 302–346 (2023).
30. M. Dianati and R. Alléaume, "Architecture of the SECOQC quantum key distribution network," in *1st International Conference on Quantum, Nano, and Micro Technologies (ICQNM)* (IEEE, 2007), p. 13.
31. "Functional requirements for quantum key distribution networks," ITU-T Recommendation Y.3801 (2020).
32. "Quantum key distribution networks—Control and management," ITU-T Recommendation Y.3804 (2020).
33. "Quantum key distribution networks—Software-defined networking control," ITU-T Recommendation Y.3805 (2021).
34. "Quantum key distribution (QKD); control interface for software defined networks," ETSI GS QKD 015 (2022).
35. "Quantum key distribution (QKD); orchestration interface for software defined networks," ETSI GS QKD 018 (2022).
36. "Quantum key distribution (QKD); protocol and data format of rest-based key delivery API," ETSI GS QKD 014 (2019).
37. "Quantum key distribution (QKD); application interface," ETSI GS QKD 004 (2020).
38. https://www.qkdnetsim.info.
39. P. Leach, M. Mealling, and R. Salz, "A universally unique identifier (UUID) URN namespace," IETF RFC 4122 (2005).
40. E. Dervisevic, M. Voznak, and M. Mehic, "Bases selection with pseudo-random functions in BB84 scheme," Heliyon **10**, e23578 (2024).
41. P. K. Tysowski, X. Ling, N. Lütkenhaus, *et al.*, "The engineering of a scalable multi-site communications system utilizing quantum key distribution (QKD)," Quantum Sci. Technol. **3**, 024001 (2018).
42. O. Maurhart, "QKD networks based on Q3P," in *Applied Quantum Cryptography* (Springer, 2010), pp. 151–171.
43. W. Dai, "Crypto++ 8.7 benchmarks," 2009, http://www.cryptopp.com/benchmarks.html.
44. S. Marksteiner and O. Maurhart, "A protocol for synchronizing quantum-derived keys in IPSEC and its implementation," in *9th International Conference on Quantum, Nano/Bio, and Micro Technologies* (2015), pp. 35–40.
45. E. Dervisevic and M. Mehic, "Overview of quantum key distribution technique within IPsec architecture," in *Proceedings of the 18th International Conference on Information Systems for Crisis Response and Management ISCRAM* (ACM, 2021).
46. M. Avesani, G. Foletto, M. Padovan, *et al.*, "A quantum key distribution network in the metropolitan area of Padova," Proc. SPIE **12446**, 124460J (2023).
47. https://www.open-qkd.eu.

103.27.10.69 2025-07-25 01:02:23